Studio Manager

Documentação Técnica e Operacional — Sistema de Gestão para Estúdios de Beleza e Estética

ARQUITETURA & ESPECIFICAÇÃO

1 Como Executar o Projeto Quando Instalado

Siga os passos abaixo para configurar o ambiente e executar o sistema localmente em modo de desenvolvimento ou produção.

📋 **Pré-requisitos**

- **Node.js**: Versão 18.x ou superior (testado e certificado no Node.js v24).
- **Gerenciador de Pacotes**: `npm` (incluso no Node), `yarn` ou `pnpm`.
- **Projeto Supabase**: Instância PostgreSQL ativa criada no supabase.com.

Passo 1: Variáveis de Ambiente

Na pasta raiz do projeto (`studio-manager`), crie o arquivo `.env.local` duplicando o modelo fornecido:

```
cp .env.example .env.local
```

Edite o arquivo `.env.local` preenchendo as chaves do seu projeto Supabase (localizadas em **Project Settings → API**):

```
NEXT_PUBLIC_SUPABASE_URL=https://seu-projeto-id.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=sua-anon-key-aqui
```

Passo 2: Configuração do Banco de Dados no Supabase

1. Acesse o painel do seu projeto no **Supabase** e clique na aba **SQL Editor**.
2. Abra o arquivo `supabase/migrations/001_initial_schema.sql` localizado no projeto.
3. Copie todo o conteúdo SQL, cole no editor do Supabase e clique em **Run**.

Este script provisionará os ENUMs de categorias e status, as tabelas `clientes`, `servicos` e `agendamentos`, índices relacionais e as políticas de segurança Row Level Security (RLS).

Passo 3: Criar Usuário de Acesso

No painel do Supabase, acesse **Authentication → Users**, clique em **Add User → Create User** e cadastre seu e-mail e senha de acesso.

Passo 4: Comandos de Execução

Modo Desenvolvimento (Dev)

Com Hot Reloading ativado:

```
npm run dev
```

Acesse: `http://localhost:3000`

Modo Produção (Build)

Compilação estática e SSR otimizada:

```
npm run build
npm run start
```

Build validado com código de saída 0.

2 Tecnologias Utilizadas (Stack Tecnológico)

CAMADA	TECNOLOGIA	PAPEL E JUSTIFICATIVA
Front-end	Next.js 14 (App Router) + React 18	Renderização otimizada (SSR/SSG), roteamento modular, páginas estáticas e suporte a Edge Middleware.
Linguagem	TypeScript 5	Tipagem estática ponta a ponta garantindo integridade de dados e ausência de erros em runtime.
Design & UI	Tailwind CSS + Shadcn / Radix UI	Design system minimalista com a paleta *Dark & Gold* (preto profundo `#0A0A0A` e ouro metálico `#C9A84C`) e acessibilidade WAI-ARIA.
Gráficos	Recharts	Exibição gráfica responsiva para faturamento mensal (BarChart) e canais de pagamento (PieChart).
Formulários	React Hook Form + Zod	Validação rigorosa de esquemas no cliente, alta performance e auto-fill dinâmico.
Back-end & DB	Supabase (PostgreSQL)	Persistência com tipagem relacional, triggers, índices otimizados e Supabase Auth integrado.
Feedback	Sonner (Toasts)	Notificações visuais e alertas de sucesso/erro integrados ao tema visual.
Deploy / CI/CD	Vercel	Deploy contínuo conectado ao repositório Git com distribuição global via Edge Network.

3 Arquitetura e Estrutura de Pastas

```
studio-manager/
├── app/
│   ├── (auth)/login/page.tsx            # Tela de login responsiva Dark & Gold
│   ├── (dashboard)/
│   │   ├── layout.tsx                   # Layout protegido (Sidebar + Header autenticado)
│   │   ├── dashboard/page.tsx           # Painel analítico financeiro (RF04)
│   │   ├── clientes/
│   │   │   ├── page.tsx                 # Listagem e busca em tempo real (RF01)
│   │   │   └── [id]/page.tsx            # Perfil do cliente e histórico de atendimentos
│   │   ├── servicos/page.tsx            # Catálogo de procedimentos agrupado (RF02)
│   │   └── agendamentos/page.tsx        # Agenda transacional com filtros (RF03)
│   ├── api/auth/callback/route.ts       # Endpoint para troca de tokens de sessão
│   ├── globals.css                      # Cores CSS Variables e tema Dark & Gold
│   └── layout.tsx                       # Layout raiz com Toaster e configurações globais
├── components/
│   ├── ui/                              # Primitivos UI (Button, Card, Dialog, Table, etc.)
│   ├── layout/                          # Sidebar de navegação e Header com logout
│   ├── clientes/                        # Modal de formulário para Clientes
│   ├── servicos/                        # Modal de formulário para Serviços
│   ├── agendamentos/                    # Modal com auto-preenchimento de valores
│   └── dashboard/                       # Cards de indicadores e gráficos Recharts
├── lib/
│   ├── supabase/                        # Clients do Supabase (Client-side e Server-side)
│   ├── queries/                         # Consultas SQL tipadas e regras de agregação
│   └── utils.ts                         # Formatadores BRL, telefone e badges semânticos
├── middleware.ts                         # Interceptador de rotas para proteção de acesso
├── supabase/migrations/                  # Scripts SQL versionados para o banco de dados
├── types/database.ts                    # Definições de tipos TypeScript do banco de dados
└── tailwind.config.ts                   # Configuração e tema visual customizado
```

4 Módulos do Sistema e Funcionalidades

🔐 Autenticação e Controle de Acesso (RNF02)

Implementado via `middleware.ts` e Supabase Auth. Rotas privadas só são renderizadas após validação do token de sessão via cookies seguros. Usuários não autenticados são automaticamente redirecionados para a rota `/login`.

👥 Módulo de Clientes (RF01 — `/clientes` e `/clientes/[id]`)

- **Gestão Completa (CRUD)**: Cadastro e edição de clientes com atributos: ID (UUID), Nome, WhatsApp e timestamp de cadastro.
- **Busca em Tempo Real**: Campo de pesquisa responsivo com filtro instantâneo por nome ou telefone.
- **Exclusão Segura**: Confirmação modal com componente `AlertDialog` antes da exclusão definitiva.
- **Perfil e Histórico (`/clientes/[id]`)**: Ao clicar no nome do cliente, abre-se a visão consolidada com quantidade total de procedimentos, total investido (R$) e a tabela completa com seu histórico de atendimentos.

🎀 Módulo de Serviços (RF02 — `/servicos`)

- **Catálogo Dinâmico**: Procedimentos organizados em cards categorizados em: *Cílios*, *Sobrancelhas* e *Depilação*.
- **Tipo de Precificação**: Suporte aos tipos *Tabela Padrão* e *Promoção* (identificado com badge dourado de destaque).
- **Preço Base**: Armazenamento do preço original utilizado como valor inicial dinâmico na agenda.

🗓️ Módulo de Agendamentos (RF03 — `/agendamentos`)

- **Agenda Transacional**:
  - Data e Horário com seletor `datetime-local`.
  - Chaves Estrangeiras vinculadas: Cliente (RF01) e Serviço (RF02).
  - **Formas de Pagamento**: *Pix*, *Cartão Débito*, *Cartão Crédito*, *Crédito Bemol* e *Dinheiro*.
  - **Status**: *Agendado*, *Confirmado*, *Concluído*, *Cancelado* e *Não Compareceu*.
  - **Local**: *Studio* ou *Domicílio*.
  - Campo para Observações adicionais.
- **Filtro Rápido**: Dropdown permitindo filtrar a listagem pelo status do atendimento.

📊 Painel Analítico / Dashboard (RF04 — `/dashboard`)

- **4 Indicadores Principais (Cards)**:
  - **Faturamento Realizado**: Somatório restrito aos registros com status `Concluído`.
  - **Faturamento Previsto**: Somatório das receitas de atendimentos `Agendado` e `Confirmado`.
  - **Volume Concluído**: Quantidade total de atendimentos realizados.
  - **Volume Pendente**: Quantidade de procedimentos agendados aguardando execução.
- **Gráfico Mensal (Recharts BarChart)**: Evolução do faturamento dos últimos 6 meses com tooltip e moeda formatada em R$.
- **Gráfico de Canais (Recharts Donut)**: Proporção de faturamento por método de pagamento.

5 Regras de Negócio e Segurança de Dados

⚡ RN01 — Preenchimento Dinâmico de Valores

No formulário de agendamento (`agendamento-form.tsx`), ao selecionar um serviço, um hook `useEffect` captura o evento e preenche automaticamente o campo **"Valor Final"** com o `preco_base` do catálogo. O campo permanece desbloqueado para que o profissional possa aplicar descontos ou taxas extras livremente.

🎯 RN02 — Validação e Acurácia de Métricas Financeiras

Todas as consultas financeiras (em `lib/queries/agendamentos.ts`) excluem obrigatoriamente qualquer agendamento com status `Cancelado` ou `Não Compareceu` . Isso impede a distorção do faturamento real e previsto.

🛡️ RNF01 — Row Level Security (RLS) no PostgreSQL

Todas as tabelas do banco possuem RLS ativada com políticas de `SELECT` , `INSERT` , `UPDATE` e `DELETE` vinculadas ao identificador do usuário autenticado (`auth.uid() = user_id`), garantindo privacidade e isolamento total dos dados.